Assignment 2 - Student Record Management System

This Assignment is only for Undergraduate students (COMP2X23).

All submitted work must be *done individually* without consulting someone else's solutions in accordance with the University's Academic Dishonesty and Plagiarism policies.

Story

You are a being asked to create a student record management system for all the students in your tutorial. For this task you will use a Binary Search Tree.

You will need to do the following:

- Define a Student class to represent student records with attributes such as student ID, name, and GPA.
- Use a binary search tree to manage student records, where each node stores a student record based on their **student ID**.
- Implement a method to insert a new student record into the binary search tree: Ensure that student records are inserted in such a way that the binary search tree property is maintained (i.e., left child's ID < parent's ID < right child's ID).
- Implement a method to delete a student record from the binary search tree based on their **student ID**: Handle different cases of deletion, such as nodes with no children, one child, or two children.
- Implement a method to search for a student record based on their **student ID**: Return the student record if found or indicate if the student record does not exist in the binary search tree.
- Validate the **correctness of the implementation** by verifying that the binary search tree property is maintained after each operation.
- Enhance the student record management system by adding functionalities such as updating student information (each of the attributes) or generating a full report of name and GPA based on the student's ID.
- Allow additional operations like:
    - Level-order traversal (**return** a list of elements level by level or by the specified level number (integer)). Level is starting from 0.
    - Finding the minimum or maximum **GPA** in the Student Record.
    - In-order traversal (**return** a list of elements of the entire tree)

You should strive to make your implementation as efficient as possible. Note that while we don't explicitly test for the efficiency of your implementation, *using inefficient implementations may cause timeouts on Ed.*

Testing

We have provided you with some test cases in the tests directory of this repository. We will be using unit tests provided with the unittest package of python.

Running Tests

From the base directory (the one with `bst.py`), run:

`python –m unittest –v test.py` OR `pytest test.py`

Marking

You will be marked using a range of public and hidden tests. Passing the given tests is not enough to get full marks. You must test your code on your own. We reserve the option of adding additional tests after the due date.